

Assignment 1: Preliminary Investigation and its activities

1.1 Problem Identification and Definition

Problem:

Readers have access to a large number of books through various online platforms. However, finding books that match their individual interests, preferences and reading history can be difficult. Traditional book search systems mainly depend on title, author or general category and may not provide sufficiently personalized recommendations.

Problem Definition:

The problem is to develop a system that analyzes the user's reading preferences and ratings and provides personalized book recommendations based on the user's interests, genres, categories and other characteristics of books.

1.2 Problem Description

There are a large number of books that can be made available to the user, but due to this large availability of books, it becomes hard for the user to find out which book he should read or not according to his preference.

Existing systems provide recommendations based upon either popularity or category, but it is possible that this process may not give the true reflection of the preferences of all users

The recommended system Booklyst helps solve this problem through gathering information about books that have been read and rated by the user, analyze this information and provide personalized recommendations.

1.3 Techniques Used to Gather Facts

1. User Observation:

Observation was made on how users go about searching for the appropriate books.

2. Interviewing:

Informal interviews were conducted with the prospective users in order to find out their expectations from book recommendations and searches.

3. Questionnaire:

Certain questions were framed based on user preferences in genres and reading, ratings and requirements from book recommendation system.

4. Analysis of Existing Systems:

Analysis was performed on the current online book sites and recommendation systems to understand their features, strengths and weaknesses.

5. Online Research:

Online research was conducted regarding the available metadata, techniques of book recommendations and databases of books.

1.4 Drawbacks of Existing System

The existing methods of finding books have the following drawbacks:

1. Recommendations may be based mainly on popularity rather than individual user preferences.
2. Users may need to manually search through a large number of books.
3. Simple genre-based recommendations may not accurately represent the user's actual interests.
4. Different users may receive similar recommendations despite having different reading preferences.
5. Existing systems may not always provide clear explanations for why a particular book is recommended.
6. User feedback may not always be effectively utilized to improve future recommendations.
7. Finding new books with similar themes or characteristics can be time-consuming.

1.5 Scope of the Proposed System

recommender system will provide a platform for users to discover books according to their personal interests and reading preferences.

The scope of the system includes:

- User registration and authentication.
- Searching and browsing books.
- Viewing detailed information about books.
- Allowing users to rate books.
- Maintaining the user's reading history.
- Identifying user preferences from previously rated books.
- Generating personalized book recommendations.
- Providing similar book recommendations.
- Allowing users to provide feedback on recommendations.
- Providing book discovery based on genres, categories and keywords.

The system will primarily focus on personalized book discovery and recommendation rather than providing complete book-reading or book-purchasing facilities.

1.6 Feasibility Study

A feasibility study was conducted to determine whether the proposed Booklyst system can be developed and implemented successfully.

1. Technical Feasibility

The system can be developed using commonly available technologies such as HTML, CSS, JavaScript, React.js, Node.js, Express.js and PostgreSQL. These technologies provide the required facilities for developing the frontend, backend, database and recommendation functionality.

The required development tools and software are easily available, making the project technically feasible.

2. Economic Feasibility

The proposed system does not require expensive hardware or specialized infrastructure. Most of the software and development tools required for the project are freely available.

Therefore, the development cost of the system is relatively low and the project is economically feasible.

3. Operational Feasibility

The system is designed to provide a simple interface through which users can search for books, rate books and receive recommendations. The system can be used without requiring advanced technical knowledge.

Therefore, the proposed system is operationally feasible.

4. Schedule Feasibility

The system can be developed in phases such as requirement analysis, database development, backend development, frontend development, recommendation implementation, testing and deployment.

Therefore, the proposed system can be completed within the available project duration.

Assignment 2: Requirement Specification

2.1 Data Requirement Of The System

system requires the following data:

User Data

- User ID
- Name
- Email address
- Password
- User preferences

Book Data

- ISBN
- Book title
- Author
- Publisher
- Publication year
- Book description
- Genre
- Category
- Keywords
- Cover image

User Interaction Data

- Book ratings
- Reading history
- Favorite books
- Reading list
- Recommendation feedback

Recommendation Data

- User preference information
- Book characteristics
- Similarity information
- Recommendation scores

The collected data will be used to generate personalized recommendations and improve the quality of book discovery.

2.2 Identify End User Of The System

The primary end user of the recommender system is:

1. Reader/User

The reader uses the system to:

- Search for books.
- View book information.
- Rate books.
- Maintain a reading list.
- Receive personalized recommendations.
- Provide feedback on recommendations.

2. Administrator

The administrator may manage the book database and monitor the overall functioning of the system.

2.3 Input Data To The System

The following data may be provided as input to the recommender system:

- User registration details.
- Login credentials.
- Book search queries.
- Book ratings.
- Books already read by the user.
- User reading preferences.
- Favorite books.
- Recommendation feedback.
- User interactions with books.

Book metadata such as ISBN, title, author, genre, category and keywords may also be imported into the system database.

2.4 Output Information From The System

system provides the following output information:

- Personalized book recommendations.
- Similar book recommendations.
- Book search results.

- Detailed book information.
- Genre and category information.
- Book ratings.
- User reading history.
- Favorite books.
- Reading list.
- Recommendation explanations.
- Personalized discovery results.

The system may also provide information regarding why a particular book has been recommended based on the user's preferences and previous interactions.

2.5 Functional/ Non functional/Processing Requirements of the System

A: Functional Requirements

system shall provide the following functions:

1. The system shall allow users to register and log in.
2. The system shall allow users to search for books.
3. The system shall display detailed information about books.
4. The system shall allow users to rate books.
5. The system shall maintain user reading history.
6. The system shall allow users to mark books as favorites.
7. The system shall analyze user ratings and interactions.
8. The system shall generate personalized book recommendations.
9. The system shall provide similar book recommendations.
10. The system shall allow users to provide feedback on recommendations.
11. The system shall update recommendations according to user preferences and interactions.

B: Non Functional Requirement

The system shall satisfy the following non-functional requirements:

Performance: The system should provide search results and recommendations within a reasonable response time.

Usability: The system should provide a simple and user-friendly interface.

Reliability: The system should handle invalid inputs and unexpected errors without losing important user data.

Security: User credentials and personal information should be protected from unauthorized access.

Scalability: The system should be capable of handling an increasing number of books and users.

Maintainability: The software should be organized into separate components so that modifications and future improvements can be implemented easily.

Availability: The system should be accessible to users whenever the application services are running.

C: Processing Requirement Of The System

The system shall perform the following processing operations:

1. Validate user input.
2. Store and retrieve book information from the database.
3. Store and analyze user ratings and reading history.
4. Identify user preferences from previous interactions.
5. Compare book characteristics with user preferences.
6. Calculate recommendation scores.
7. Rank candidate books according to their recommendation scores.
8. Remove books that the user has already read or rejected.
9. Generate a final list of recommended books.
10. Update the recommendation results when new user feedback becomes available.

Assignment 3: Database Design

3.1 Identify the Entities and the Attributes

USER

Attribute	Description	Key / Constraint
user_id	Unique identification number of the user	PK
name	Name of the user	
email	Email address	UNIQUE
password_hash	Hashed password	

BOOK

Attribute	Description	Key / Constraint
book_id	Unique identification number of the book	PK
isbn	International Standard Book Number	UNIQUE
title	Title of the book	
author_name	Name of the author	
publisher_name	Name of the publisher	
publication_year	Year of publication	
description	Description of the book	
cover_url	Book cover image URL	

GENRE

Attribute	Description	Key / Constraint
genre_id	Unique identification number of the genre	PK
name	Name of the genre	UNIQUE
description	Description of the genre	

KEYWORD

Attribute	Description	Key / Constraint
keyword_id	Unique identification number of the keyword	PK
name	Keyword associated with a book	UNIQUE

RATING

Attribute	Description	Key / Constraint
rating_id	Unique rating identification number	PK
user_id	User who provides the rating	FK
book_id	Book being rated	FK
overall_rating	Overall rating	
plot_rating	Rating for plot/story	
character_rating	Rating for characters	
writing_rating	Rating for writing style	
pacing_rating	Rating for pacing	
worldbuilding_rating	Rating for world building	

BOOK_GENRE

Attribute	Description	Key / Constraint
book_id	Book identifier	PK, FK
genre_id	Genre identifier	PK, FK

BOOK_KEYWORD

Attribute	Description	Key / Constraint
book_id	Book identifier	PK, FK
keyword_id	Keyword identifier	PK, FK

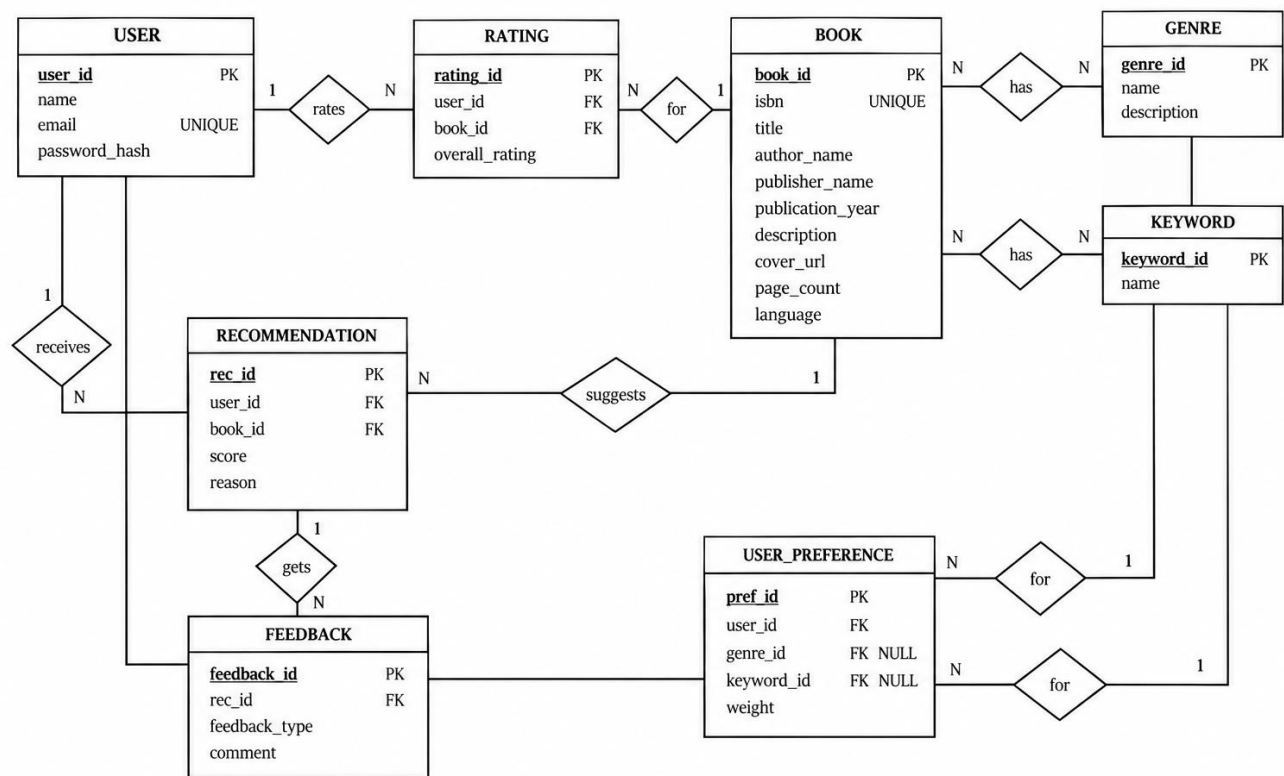
RECOMMENDATION

Attribute	Description	Key / Constraint
recommendation_id	Unique recommendation identification number	PK
user_id	User receiving the recommendation	FK
book_id	Recommended book	FK
score	Recommendation score	
reason	Reason for recommendation	

FEEDBACK

Attribute	Description	Key / Constraint
feedback_id	Unique feedback identification number	PK
recommendation_id	Related recommendation	FK
feedback_type	Type of feedback	
comment	Optional comment	

3.2 E-R Diagram



Relationship	Description	Cardinality
USER – RATING	A user can provide many ratings; each rating belongs to one user.	1:N
BOOK – RATING	A book can receive many ratings; each rating belongs to one book.	1:N
BOOK – GENRE	A book can have many genres and a genre can belong to many books.	N:M
BOOK – KEYWORD	A book can have many keywords and a keyword can belong to many books.	N:M
USER – RECOMMENDATION	A user can receive many recommendations.	1:N
BOOK – RECOMMENDATION	A book can be recommended to many users.	1:N
RECOMMENDATION – FEEDBACK	A recommendation can receive feedback.	1:N

3.3 Identifying All Tables, Fields, and Relationships Between Tables

Table	Fields
USER	user_id (PK), name, email (UNIQUE), password_hash, created_at

BOOK	book_id (PK), isbn (UNIQUE), title, author_name, publisher_name, publication_year, description, cover_url
GENRE	genre_id (PK), name (UNIQUE), description
KEYWORD	keyword_id (PK), name (UNIQUE)
RATING	rating_id (PK), user_id (FK), book_id (FK), overall_rating, plot_rating, character_rating, writing_rating, pacing_rating, worldbuilding_rating, review_text, created_at
BOOK_GENRE	book_id (PK, FK), genre_id (PK, FK)
BOOK_KEYWORD	book_id (PK, FK), keyword_id (PK, FK)
RECOMMENDATION	recommendation_id (PK), user_id (FK), book_id (FK), score, reason, created_at, status
FEEDBACK	feedback_id (PK), recommendation_id (FK), feedback_type, comment, created_at

Foreign-Key Relationships

Foreign Key	References
RATING.user_id	USER.user_id
RATING.book_id	BOOK.book_id
BOOK_GENRE.book_id	BOOK.book_id
BOOK_GENRE.genre_id	GENRE.genre_id
BOOK_KEYWORD.book_id	BOOK.book_id
BOOK_KEYWORD.keyword_id	KEYWORD.keyword_id
RECOMMENDATION.user_id	USER.user_id
RECOMMENDATION.book_id	BOOK.book_id
FEEDBACK.recommendation_id	RECOMMENDATION.recommendation_id

3.4 Normalize Database

First Normal Form (1NF)

The database satisfies 1NF because each field contains an atomic value and there are no repeating groups. Multiple genres and keywords are not stored as comma-separated values in BOOK. Their relationships are stored separately in BOOK_GENRE and BOOK_KEYWORD.

Second Normal Form (2NF)

The database satisfies 2NF because it is in 1NF and every non-key attribute depends on the complete primary key. The junction tables use composite keys consisting of both related identifiers.

Third Normal Form (3NF)

The database satisfies 3NF because it is in 2NF and non-key attributes do not depend on other non-key attributes. For example, genre names are stored in GENRE rather than repeatedly in BOOK.

Advantages of Normalization

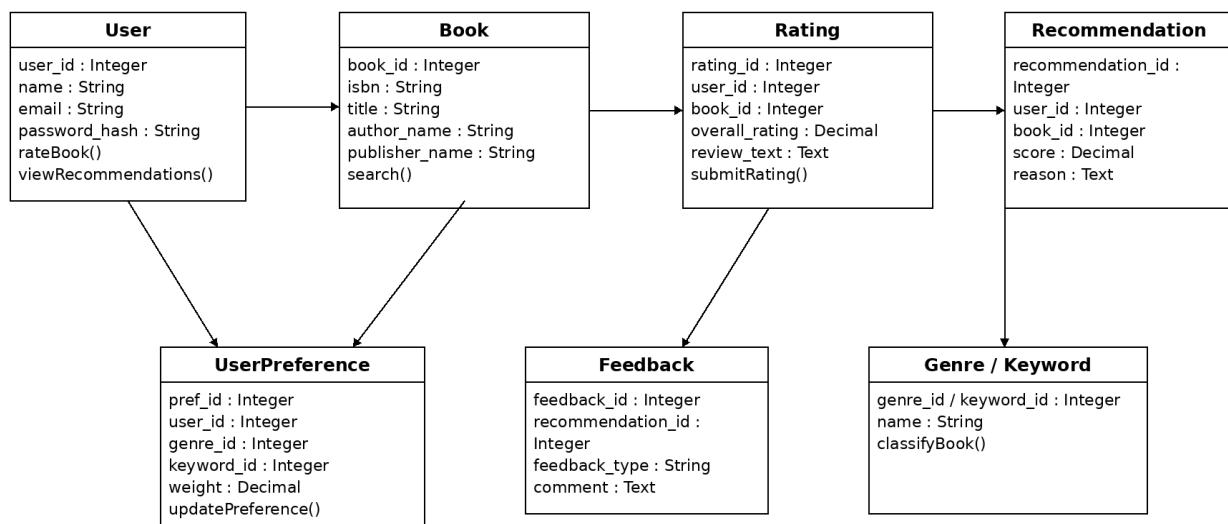
- Reduces unnecessary duplication of data.
- Improves consistency and data integrity.
- Reduces insertion, update and deletion anomalies.
- Makes relationships easier to query using SQL JOIN operations.
- Provides a suitable structure for implementing the recommendation algorithm.

Assignment 4: System Design

4.1 Class Diagram

Classes are User, Book, Rating, Recommendation, UserPreference, Feedback, and Genre/Keyword. The relationships show how these classes interact during book rating and recommendation generation.

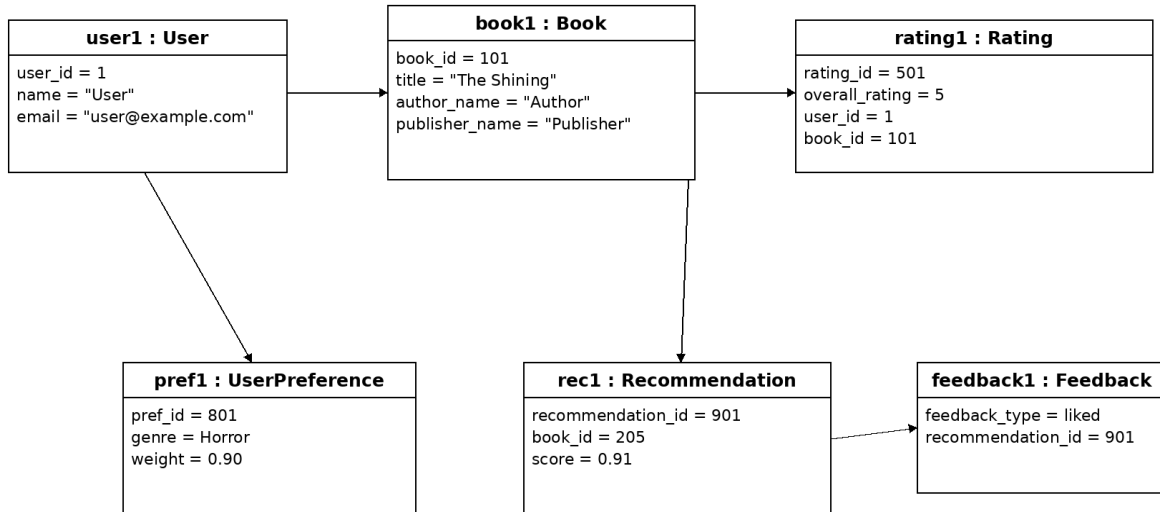
4.1 Class Diagram



4.2 Object Diagram

The object diagram represents an example snapshot of the system at a particular point in time. It shows actual objects such as a user, a book, a rating, a user preference, a recommendation, and feedback, together with representative values.

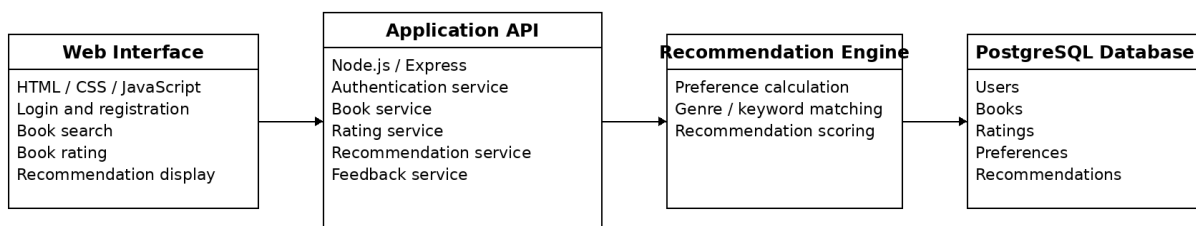
4.2 Object Diagram



4.3 Component Diagram

The frontend communicates with the Node.js/Express API. The API handles authentication, book services, ratings, recommendations and feedback. The recommendation engine calculates preferences and recommendation scores, while PostgreSQL stores the application data.

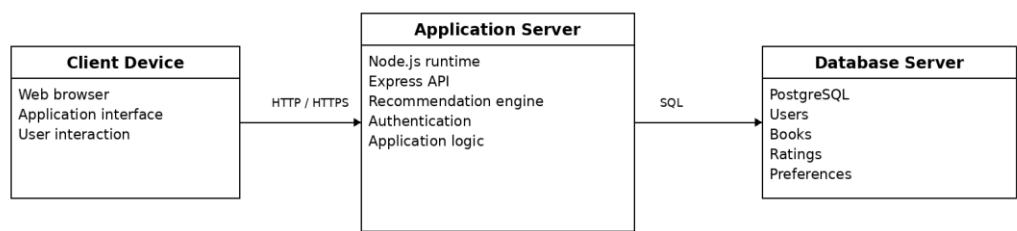
4.3 Component Diagram



4.4 Deployment Diagram

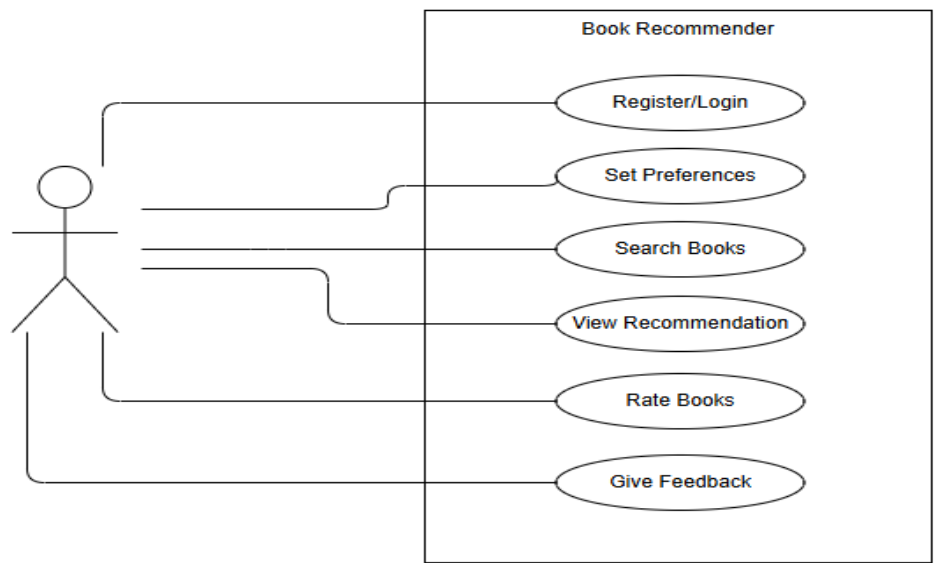
The client device runs the recommender system’s frontend in a web browser. The application server runs Node.js, Express and the recommendation engine. The database server stores the PostgreSQL database.

4.4 Deployment Diagram



4.5 Use Case Diagram

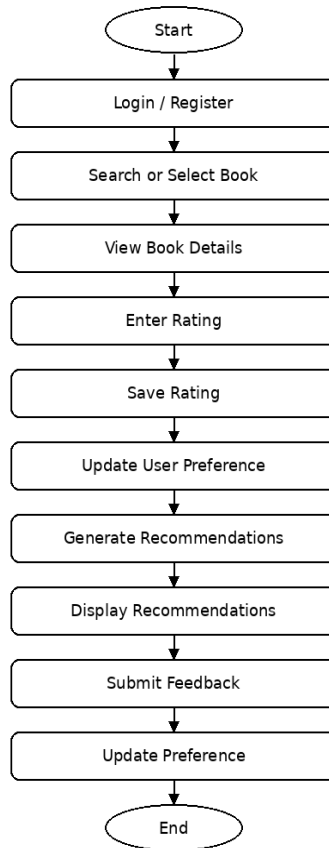
The use case diagram represents the functions available to the user. The user can register or log in, search for books, view book details, rate books, view person



4.6 Activity Diagram

The activity diagram represents the workflow of the Book Recommendation System from user login through book selection and rating to preference calculation and recommendation generation. Recommendation feedback can subsequently update the user's preferences.

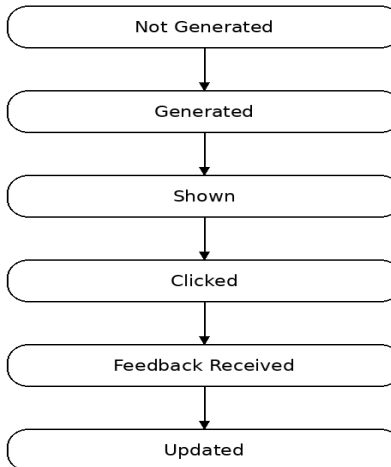
4.6 Activity Diagram



4.7 State Chart Diagram

The state chart diagram represents the changing state of a recommendation during its lifecycle. A recommendation can move from not generated to generated, shown, clicked, feedback received, and finally updated.

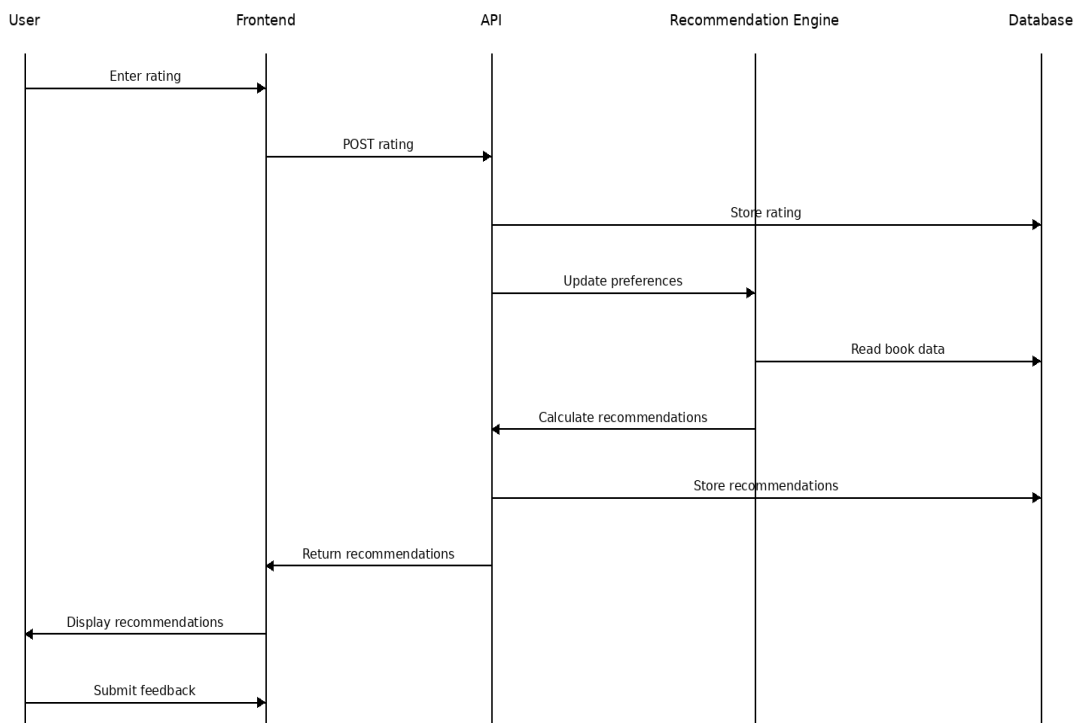
4.7 State Chart Diagram



4.8 Sequence Diagram

The sequence diagram represents the order of communication between the user, frontend, API, recommendation engine and database. It shows how a rating is submitted, stored, used to update preferences, and followed by generation and display of recommendations.

4.8 Sequence Diagram



4.9 Collaboration Diagram

The collaboration diagram represents the interaction between the major objects and components using numbered messages. It emphasizes how the user, frontend, API, recommendation engine and database cooperate to process a rating and return recommendations.

4.9 Collaboration Diagram

